



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: PABLO SEBASTIAN NOYOLA
TORRES

N° de Cuenta: 320023213

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026 - 1

FECHA DE ENTREGA LÍMITE: 08 DE MARZO DE 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

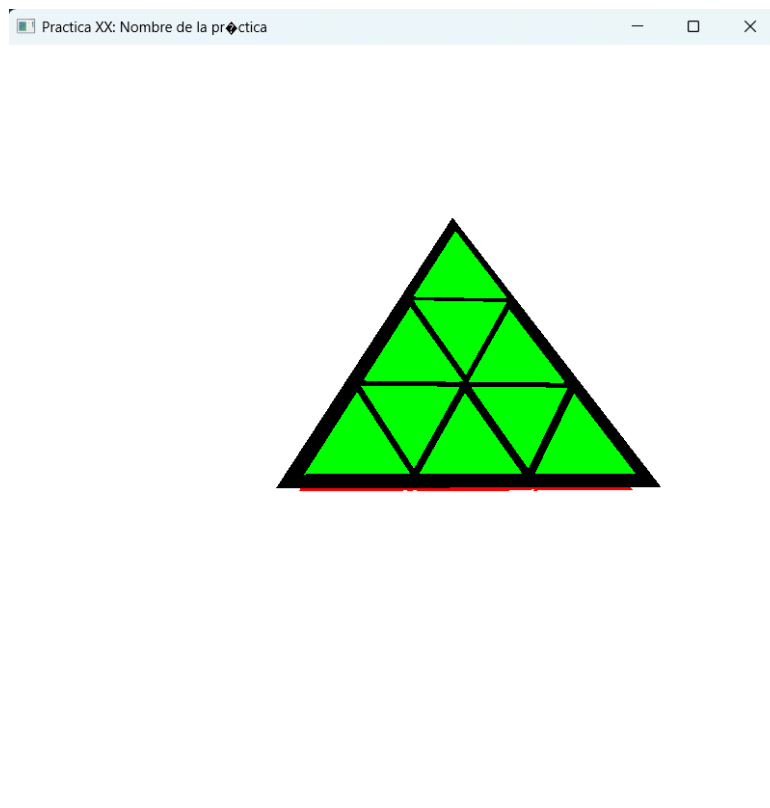
1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

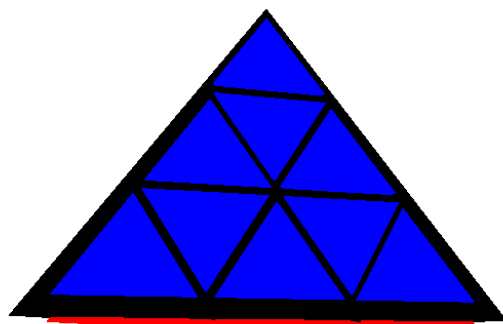
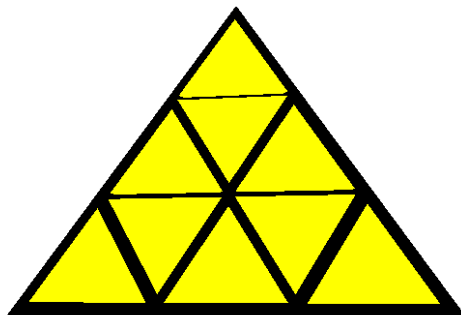
1.- Generar una pirámide rubik (pyraminx) de 9 pirámides por cara.

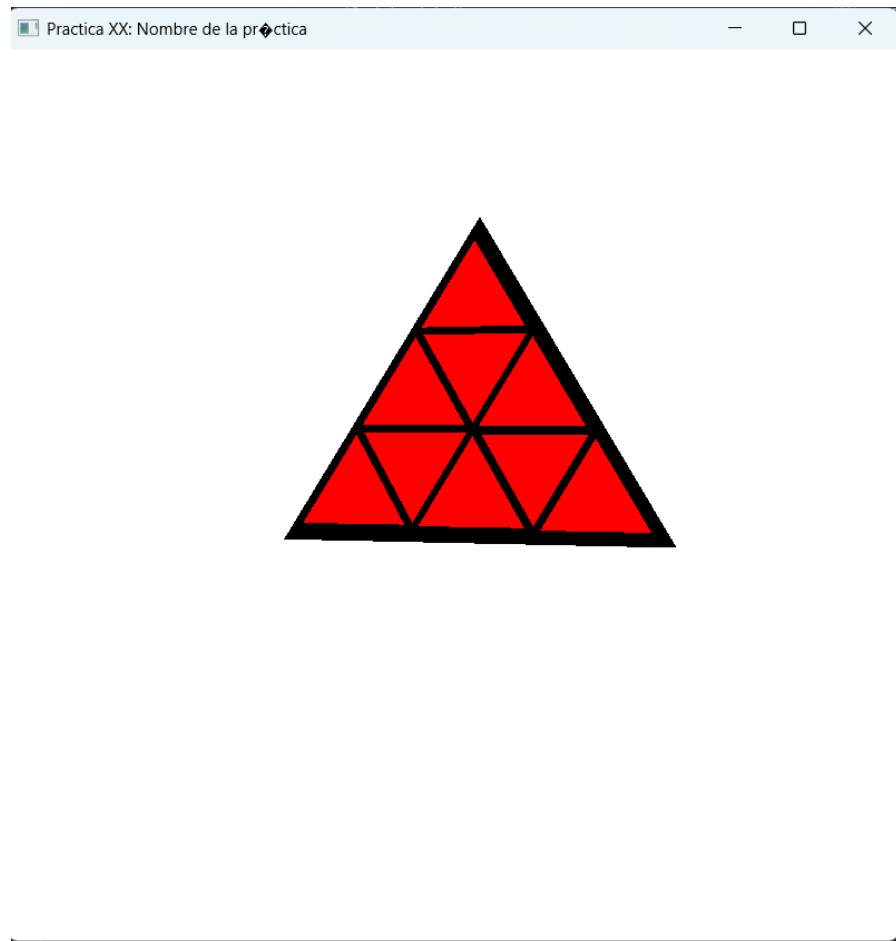
Cada cara de la pyraminx que se vea de un color diferente y que se vean las separaciones entre instancias (las líneas oscuras son las que permiten diferenciar cada pirámide pequeña)

Agregar en su documento escrito las capturas de pantalla necesarias para que se vean las 4 caras de toda la pyraminx o un video en el cual muestra las 4 caras.

Capturas de ejecución:







Video de ejecución: <https://www.youtube.com/watch?v=9QncMOsMCfY>

Código de ejercicio:

```

void CrearPiramideTriangularParaRotar()
{
    unsigned int indices_piramide_triangular[] = {
        0,1,2,
        1,3,2,
        3,0,2,
        1,0,3
    };

    GLfloat vertices_piramide_triangular[] = {
        0.0f,  0.0f,  0.5f,    //0
        0.0f,  0.0f,  -0.5f,   //1
        0.29f, -0.82f, 0.0f,    //2
        -0.67f, -0.55f, 0.0f,    //3
    };

    Mesh* obj1 = new Mesh();
    obj1->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
    meshList.push_back(obj1);
}

```

```

int main()
{
    mainWindow = Window(800, 800);
    mainWindow.Initialise();
    //Cilindro y cono reciben resolución (slices, rebanadas) y Radio de circunferencia de la base y tapa

    CrearCubo(); //índice 0 en MeshList
    CrearPiramideTriangular(); //índice 1 en MeshList
    CrearCilindro(5, 1.0f); //índice 2 en MeshList
    CrearCono(25, 2.0f); //índice 3 en MeshList
    CrearPiramideCuadrangular(); //índice 4 en MeshList
    CrearPiramideTriangularParaRotar(); //índice 5 en MeshList
    CreateShaders();
}

```

```

//Pyraminx-----
//Transformación Global (E, R, T)
glm::mat4 globalRotation = glm::mat4(1.0f);
globalRotation = glm::translate(globalRotation, glm::vec3(0.0f, 0.0f, -4.0f));
globalRotation = glm::rotate(globalRotation, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
globalRotation = glm::rotate(globalRotation, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
globalRotation = glm::rotate(globalRotation, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));

glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));

//Piramide triangular negra de base-----
model = globalRotation; // Empezamos desde la rotación global
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[1]->RenderMesh();

//Cara Roja-----
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));

```

```

//Piramide 1-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.0f, -0.01f, 0.2f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 2-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.15f, -0.01f, -0.0725f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 3-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.15f, -0.01f, -0.0725f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 4-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.3f, -0.01f, -0.34f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 5-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.0f, -0.01f, -0.34f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 6-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.3f, -0.01f, -0.34f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 7-----
model = globalRotation;

```

```

//Piramide 7-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.0f, -0.01f, -0.09f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 8-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.15f, -0.01f, -0.36f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 9-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.15f, -0.01f, -0.36f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```
//Cara azul-----
glm::mat4 sideRotation = glm::rotate(globalRotation, glm::radians(120.0f), glm::vec3(0.0f, 1.0f, 0.0f));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

```
//Piramide 1-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.03f, 0.04f, 0.21f)); // Subi ligeramente Y a 0.04f
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();
```

```
//Piramide 2-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.175f, 0.04f, -0.055f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();
```

```
//Piramide 3-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.32f, 0.04f, -0.31f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();
```

```
//Piramide 4-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.18f, 0.29f, -0.215f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();
```

```
//Piramide 5-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.03f, 0.29f, 0.03f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();
```

```
//Piramide 6-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.025f, 0.55f, -0.135f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();
```

```
//Piramide 7-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.1f, 0.525f, -0.15f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, glm::radians(-30.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMesh();
```

```
//Piramide 8-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.1f, 0.27f, 0.02f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, glm::radians(-30.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMesh();
```

```
//Piramide 9-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.25f, 0.27f, -0.24f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, glm::radians(-30.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMesh();
```

```

//Cara Verde-----
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));

//Piramide 1-----
model = globalRotation; // Reiniciamos a la rotación global
model = glm::translate(model, glm::vec3(-0.03f, 0.035f, 0.21f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

//Piramide 2-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.175f, 0.035f, -0.055f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

//Piramide 3-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.32f, 0.035f, -0.31f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 4-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.18f, 0.285f, -0.215f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

//Piramide 5-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.03f, 0.285f, 0.03f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

//Piramide 6-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.025f, 0.54f, -0.135f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 7-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.1f, 0.525f, -0.15f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, glm::radians(-150.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMesh();

//Piramide 8-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.1f, 0.27f, 0.02f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, glm::radians(-150.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMesh();

//Piramide 9-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.25f, 0.27f, -0.24f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, glm::radians(-150.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMesh();

```



```

//Cara Amarilla-----
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));

// Piramide 1-----
model = globalRotation; // Reiniciamos a la rotación global
model = glm::translate(model, glm::vec3(-0.31f, 0.03f, -0.375f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

//Piramide 2-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.0f, 0.03f, -0.375f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

//Piramide 3-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.31f, 0.03f, -0.375f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

```

```

//Piramide 4-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.15f, 0.285f, -0.28f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

//Piramide 5-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.15f, 0.285f, -0.28f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

//Piramide 6-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.0f, 0.54f, -0.19f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();

//Piramide 7-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.0f, 0.525f, -0.34f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, glm::radians(-270.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMesh();

```

```

//Piramide 8-----
model = globalRotation;
model = glm::translate(model, glm::vec3(0.15f, 0.27f, -0.43f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, glm::radians(-270.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMesh();

//Piramide 9-----
model = globalRotation;
model = glm::translate(model, glm::vec3(-0.15f, 0.27f, -0.43f));
model = glm::scale(model, glm::vec3(0.27f, 0.27f, 0.27f));
model = glm::rotate(model, glm::radians(-270.0f), glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[5]->RenderMesh();

```

Para este ejercicio, basándome en lo que ya hemos trabajado en las prácticas anteriores y ejercicios de clase, lo que hice fue adaptar las coordenadas de los vértices de las mallas y las pirámides al código proporcionado, ajustando los parámetros de transformación para posicionarlas correctamente. Modifiqué los

colores mediante uniformes y utilicé una matriz de rotación global para que todas las piezas, desde la base negra hasta las caras de colores, mantuvieran la coherencia visual al rotar el objeto completo en el espacio 3D.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

- Cálculo manual de coordenadas: Al crear las pirámides y el cubo, el principal reto fue definir los vértices exactos para que las caras cerraran perfectamente.
- Jerarquía de transformaciones: Al principio, las pirámides pequeñas no seguían la rotación del objeto completo. Esto lo solucioné aplicando la `globalRotation` como base de la matriz `model` antes de cualquier traslación o escala local, asegurando que todos los objetos heredaran el movimiento global.
- Inversión de normales y caras: Al rotar algunas piezas 180° , las caras desaparecían o se veían raras por el orden de los vértices. Para solucionar esto y mantener el orden de los índices en sentido contrario a las agujas del reloj (CCW), implementé la función `CrearPiramideTriangularParaRotar()`, la cual me permitió manejar de forma independiente la estructura de estas mallas y asegurar que el renderizado fuera correcto sin importar la orientación.
- Superposición de caras (Z-Buffer): En algunas pruebas las caras traseras se dibujaban sobre las delanteras. Lo corregí asegurándome de tener activo el `glEnable(GL_DEPTH_TEST)` en la inicialización de la ventana.

3.- Conclusión:

Tras la realización de esta tercera práctica considero que fue de gran ayuda para el trabajo con figuras geométricas en 3D, ya que me permitió comprender mejor cómo estructurar mallas complejas y aplicar jerarquías de transformación mediante matrices globales y locales. Sin embargo, aunque logré solucionar problemas específicos de rotación y visualización implementando funciones propias para el manejo de vértices, aún considero que debo mejorar en mi manejo de OpenGL para optimizar el renderizado y profundizar en el control avanzado de los estados de la GPU.

1. Bibliografía en formato APA

García, A. (2022). Fundamentos de computación gráfica y visualización. Editorial Universitaria.

Lengyel, E. (2012). Mathematics for 3D Game Programming and Computer Science (3rd ed.). Course Technology PTR.

Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). OpenGL Programming Guide: The Official Guide to Learning OpenGL (8th ed.). Addison-Wesley Professional.

G-Truc Creation. (n.d.). OpenGL Mathematics (GLM) Project Documentation.
<https://glm.g-truc.net/>

Khronos Group. (s.f.). OpenGL Wiki: Viewing and Transformations.
https://www.khronos.org/opengl/wiki/Viewing_and_Transformations

LearnOpenGL. (s.f.). Camera and Coordinate Systems.
<https://learnopengl.com/Getting-started/Camera>

Song Ho Ahn. (2021). OpenGL Transformation Matrix.
http://www.songho.ca/opengl/gl_transform.html